18.C21 Practice Midterm
Spring 2026 (Johnson & Marzouk)

NAME: .....

- The practice exam is likely to be both **harder** and **longer** than the real exam.
- You will have **50 minutes** after your exam begins.
- Write your **NAME** above *and* at the top of *every* page. Please do it as soon as you begin the exam, and check again before returning it.
- Only write on *one* side of each sheet. Additional paper will be provided upon request.
- You can bring **1 letter-sized sheet of notes** (both sides if you want), though it is unlikely to help much.
- **Please allocate your time carefully**, to make sure you get to work on all parts. Take a quick look at the *whole exam*, before starting working on your first problem.
- **Show your work / justify your answers.** Wrong answers may still receive partial credit *if* the reasoning was somewhat correct.
- If you absolutely believe there is something wrong with a problem, please ask. No hints, other than the ones already written in the exam, will be given.

Problem 1

Recall that double-precision (`Float64`) floating-point numbers store about 15–16 decimal significant digits, and can represent magnitudes from around $\approx 10^{-300}$ to around $\approx 10^{300}$ (after which they under/overflow, respectively).

1. Many libraries, including SciPy, provide a function $\text{logsumexp}(x)$ which takes a vector $x \in \mathbb{R}^n$ and computes the mathematical function:

$$\text{logsumexp}(x) = \log\left(\sum_{i=1}^{n} e^{x_i}\right).$$

   Explain why, for accuracy reasons (ignoring performance), this function should *not* be implemented naively using the above formula (e.g. *don't* do `log(sum(exp.(x)))` in Julia or `numpy.log(numpy.sum(numpy.exp(x)))` in Python). In fact, there are *two* extremes in which the naive formula goes horribly wrong and gives no correct digits in floating-point arithmetic. What are they? (Hint: think about $\max_i x_i$).

2. Outline (as pseudo-code) the steps of an modified algorithm to compute the same function without the catastrophes identified in the previous part. (Hint: factor something out of the sum, and remember the identity $\log(ab) = \log a + \log b$.)

Problem 2

The "error" function $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$ is a standard special function (provided by many libraries, such as `scipy.special.erf`) that computes *part* of a Gaussian integral. It is plotted in Fig. 1 and has the property that $\mathrm{erf}(x) \to 1$ exponentially fast as $x \to \infty$, because $\int_0^\infty e^{-t^2}\, dt = \sqrt{\pi}/2$ (the famous "Gaussian integral").

1. The same libraries *also* provide a function $\mathrm{erfc}(x) = 1 - \mathrm{erf}(x)$, the "complementary error function." Why? Aren't erf and erfc redundant? What information would you potentially lose if you simply subtract erf from 1 yourself? Conversely, given erfc, why do you still need erf if you want don't want to lose information for some inputs, rather than using $\mathrm{erf}(x) = 1 - \mathrm{erfc}(x)$?

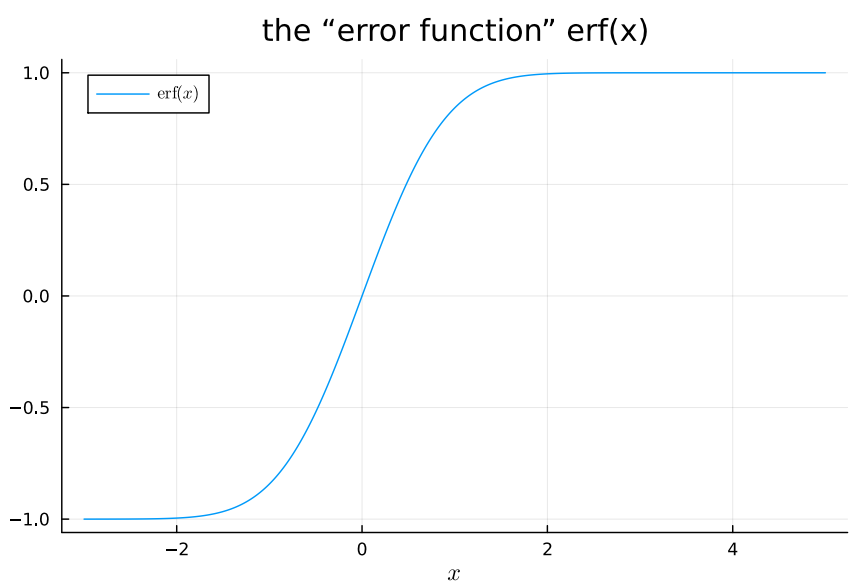

Figure 1: Plot of the error function $\mathrm{erf}(x)$.

2. For large $x \gg 1$, the error function is approximately $\mathrm{erf}(x) \approx 1 - e^{-x^2}/(x\sqrt{\pi})$. Use this to compute the relative condition number $\kappa_r(\mathrm{erfc}, x)$ for large $x \gg 1$. And by inspection of the plot (of erf), what is the relative condition number of erfc (not erf) as $x \to 0$?

   Comment: These results explain why there should be an algorithm to compute erfc accurately when $\mathrm{erf}(x)$ is close to 1, despite the problem you identified with the naive formula in the previous part.

   (Recall that for any differentiable scalar function $f(x)$, $\kappa_r(f, x) = |f'(x)|\frac{|x|}{|f(x)|}$. Also recall the fundamental theorem of calculus, saying that the derivative is the inverse of integration: $(\int^x g(t)\,dt)' = g(x)$.)

3. erf is the integral of a smooth function e^{-t^2}, so you can compute it yourself for any given x very accurately with quadrature schemes like Clenshaw–Curtis (with errors that decrease at least exponentially fast with the number of quadrature points t). However, if we wanted to evaluate the function $\mathrm{erf}(x)$ for many different x values (and let's pretend you don't have a library function like `scipy.special.erf` available), say for $x \in [-10, 10]$, outline (as a sequence of steps) how you might construct an approximation $a(x)$ for $\mathrm{erf}(x)$ on $[-10, 10]$ such that (after the construction) the resulting $a(x)$ can then be evaluated quickly and accurately (to, say, at least 10 digits) for any given value of x, probably more quickly than re-doing quadrature for each x?

Problem 3

In Fig. 2(left) are shown three functions $f_1(x)$, $f_2(x)$, and $f_3(x)$ on $x \in [0, 1]$. On the right are shown the convergence of Clenshaw–Curtis quadrature to compute $\int_0^1 f_i(x)\,dx$ for $n+1$ point versus n for these three functions, labeled A, B, C, but we do *not* tell you which of f_1, f_2, f_3 are referred to by A, B, C.

1. Which function in Fig. 2(left) correspond to which convergence curve in Fig. 2(right), and why? Give a few words of justification, no formal analysis required.

2. For the two functions that are converging slowly in Fig. 2(right), explain how you might modify your integration algorithm (given a function that does Clenshaw–Curtis quadrature on any desired interval $[a, b]$ for any n) to get fast convergence for these two cases.

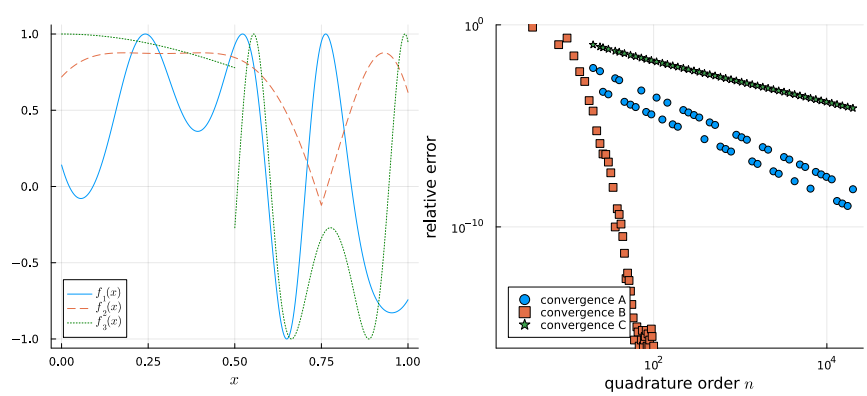

Figure 2: Plot of three functions $f_1(x)$, $f_2(x)$, and $f_3(x)$ on $x \in [0, 1]$ (left), and the convergence versus Clenshaw–Curtis quadrature order n ($n+1$ points) for these three functions, but in an unspecified order.

Problem 4

In problem set 1, you found that the quadratic curve $q(x)$ that goes through the three points $(0, f(0))$ and $(\pm\Delta x, f(\pm\Delta x))$ is:

$$q(x) = \frac{1}{2\Delta x^2}\left[f(-\Delta x)x(x-\Delta x) - 2f(0)(x^2 - \Delta x^2) + f(+\Delta x)x(x+\Delta x)\right]$$

Explain how you would use this to **derive a quadrature rule** to estimate the integral

$$\int_{-\Delta x}^{+\Delta x} f(x)\,dx \approx w_1 f(-\Delta x) + w_2 f(0) + w_3 f(+\Delta x)\,,$$

similar to how we derived the trapezoidal rule in class. **Give the weights** w_1, w_2, w_3. (This is called "Simpson's rule.")

Problem 5

Suppose that we have 100 measurements (p_k, v_k) of the volume v of a gas vs. its pressure p, and we want to fit it to a function of the form $v(p) = \frac{c_1}{p} + c_2$ for unknown constants c_1, c_2. Write down a 2×2 system of equations you could solve to find c_1, c_2 in order to minimize the sum of the squared errors $\sum_k [v(p_k) - v_k]^2$. You can write your answer (left- and right-hand sides) as products of matrices and/or vectors, as long as you specify what each term is (in terms of the unknowns c_1, c_2 and/or the data $p_1, \ldots, p_{100}$ and $v_1, \ldots, v_{100}$).

(Recall that for least-square problems $\min_x \|Ax - b\|_2$ we derived the normal equations $A^T Ax = A^T b$ for the minimizer x, though we will see that there are often better ways to solve such problems.)

Problem 6

The $\mathrm{sinc}(x)$ function is defined by $\mathrm{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$ and shows up in many important applications in signal-processing and elsewhere. It is plotted in Fig. 3, below, along with the numerator $\sin(\pi x)$ for comparison.

While the $\sin(\pi x)$ function has extrema (minima/maxima) at $x = 0.5, 1.5, 2.5, \ldots$, the extrema of the sinc function are slightly shifted—for example, the first minimum, and in fact the global minimum, of sinc is at $x \approx 1.43$, shown by a black dot in the figure.

Outline the steps of an efficient algorithm to rapidly compute $\min_x \mathrm{sinc}(x) \approx \mathrm{sinc}(1.43)$ to very high accuracy (say, more than 12 digits), making use (somehow) of Newton steps (recall: to solve $f(x) = 0$ we take steps $x \to x - f'(x)^{-1} f(x)$).

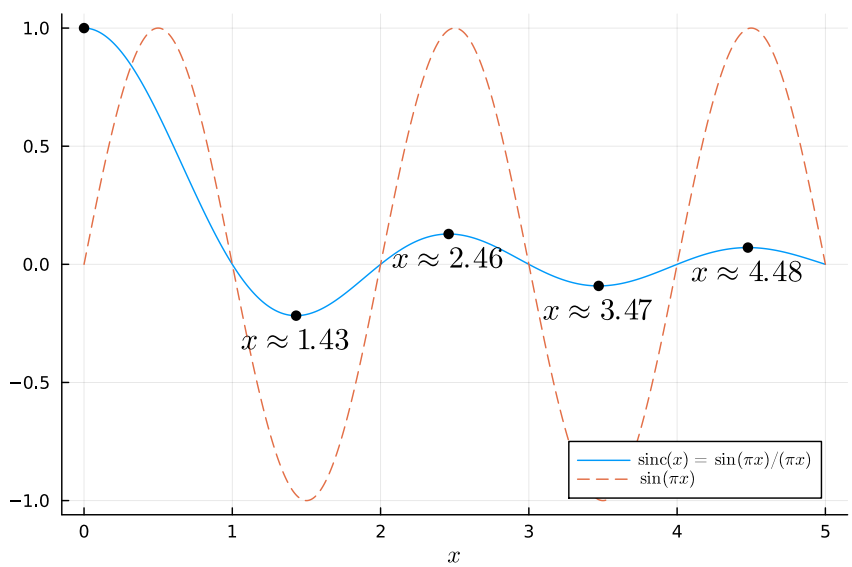

Figure 3: Plot of $\mathrm{sinc}(x)$, along with its numerator $\sin(\pi x)$. Extrema of the sinc function are indicated by labelled black dots.